L1 – Introduction to MakeCode

Robotics & Control • MYP Design Year 8 • BBC micro:bit Unit 1
Making, *Lesson B (adapted)*

Design 8B • Mr. Rawson • 18 students

8B	Thu 3 Sep, P5, 14:05–15:00	**Next**	Thu 10 Sep, P6 — **7 days**

Source BBC micro:bit Unit 1, Lesson B — **Lesson A skipped**, **micro:pet dropped** (spec too thin: three of its five requirements concern the cardboard enclosure). Do not reintroduce either.

Artefact A published MakeCode project that spells the student's name letter by letter (`show leds` frames + `pause`), cycling inside a `repeat` loop, triggered by `on button A pressed` — input → process → output, not just LED output.

To print **18** handouts (one-page quick reference, unchanged from 8A) and **18** homework sheets — the **8B** homework sheet, dated Thu 10 Sep. The 8A sheet says 11 Sep and is wrong for this class.

Homework Open-ended: build something of their own in MakeCode, publish it, submit the sharing link to **Google Classroom** before Thu 10 Sep. Post the assignment in ManageBac against the **8B** section, linked out to Classroom — a separate posting from 8A's, with its own date. Seven days, not eleven; the sheet is written to survive the gap without help.

Kit micro:bits + micro-USB cables, one set per group. Reuse Monday's numbering from the 8A inventory, then extend it. **18 students — two more than 8A's 16**, so this is the bigger class and needs *more* kit than Monday, not less. **[[ALEX: decide — does board stock cover 18 — if not, group sizes go up and the download-loop checkpoint takes longer]]**

By the end of the lesson students can:

- move a program from the MakeCode editor onto a physical micro:bit, and do it again unaided — the download loop is the skill;
- build an animation from `show leds` frames and `pause`, and wrap it in a `repeat` loop;
- trigger a program from an input (`on button A pressed`) and say which part of their program is input, which is process, which is output;
- name, save and **publish** a project, and keep the sharing link — homework is submitted as that link.

Timing

Time	Phase	What happens
0–5	Do Now	On the board as they arrive: *"What is a computer program?"* (a set of instructions that tell a computer what to do). Take answers, then frame the hour: today every one of you gets a computer, puts your own program on it, and publishes it.
5–13	Kit out + **inventory**	Hand out boards and cables *yourself*, one at a time, logging each against the inventory table (page 2): number, V1 or V2, condition, who has it. Safety in one breath: dry hands, hold by the edges, no metal across the circuits. Students connect by USB while you circulate.
13–20	MakeCode + first frame	makecode.microbit.org → New Project → name it `name-badge`. Thirty-second tour: simulator left, toolbox middle, workspace right. Drag `show leds` into `on start`; draw a single letter or shape. The simulator shows it immediately — say out loud that the simulator is not the finish line.
20–30	**First download loop**	The whole class gets one frame onto the physical board. Download button → .hex file → onto the micro:bit. Same devices and same route as Monday — do not re-improvise it at the front of the room. **Checkpoint: no one moves on until every board is lit.** Fast finishers help neighbours — that is the job, not a treat.
30–40	Build the name	One `show leds` frame per letter, `pause 500 ms` after each. Then Loops drawer: wrap the lot in `repeat 4 times` so the name cycles. Circulate for the two stalls: pause missing (letters blur past) and pause in seconds-not-ms.
40–46	Add the input	Drag the blocks out of `on start` into `on button A pressed` (Input drawer). Now nothing happens until the button is pressed — ask them why that is better, and name the frame: button = input, loop = process, LEDs = output. This framing is the hook Unit 2 (Algorithms) hangs on.
46–55	**Publish** + homework	Share → Publish project → copy the link. Every student writes their link on the handout *before* kit comes in — the link is case-sensitive and not searchable; lost link means republish. Issue the homework sheet, name the due date (before Thu 10 Sep, on Google Classroom), collect kit against the inventory.

Questions to ask

1. Your program lives in three places by the end of today. Where? (editor, the downloaded file, the board — and the published link makes four.)
2. Unplug the board and power it again. Why does the program still run?
3. What happens to the old program when you download a new one? (Replaced — the board holds one at a time. Not a bug.)
4. The simulator shows your name but the board shows nothing. What do you check, in order? (Cable seated, did the download finish, did the file actually reach the board.)
5. Which block is the input? The process? The output? What could you swap each one for?

Common misconceptions

- **"It works" meaning the simulator.** The artefact is the physical board running the program. Insist on the distinction from the first frame.
- **Download button ≠ program on the board.** In the browser flow the .hex lands in Downloads and must still be copied across. Expect several students stuck exactly here.
- `pause` **is in milliseconds.** A student types 5 expecting five seconds; another types 5000 per letter and thinks the board is broken.
- `show string` **does the whole name in one block.** Someone will find it. Fine as an *extra* — but the target is composing frames and a loop, so it does not replace the build. Ask them what `show string` must be doing underneath.
- **The sharing link is case-sensitive and cannot be searched for.** Written on the handout, in class, before packing up — or the homework submission route is gone.

Differentiation

Support: the handout is the self-serve reference — point at the step number rather than re-teaching; straight-line letters (E, L, T, I) are much easier to draw on a 5×5 grid than curves; seat confident finishers next to strugglers for the download loop.
Extend: `on button B pressed` doing something different; shrink the pauses until the name is unreadable and find the readable limit; a second animation from the Icons blocks; compare their frame-built name with `show string` and explain the difference.

Assessment consequence — flagged, not solved

Dropping the micro:pet removes this unit's design-cycle task. The Assessment Map lists Unit 01 as one of only **three** design-cycle moments in the year (01 lite A–D, 06 A+B, 13 C+D) — the "every criterion seen twice" structure loses its first sighting of **A, B and D**. As now taught, Unit 01 yields Criterion C–type skills evidence only, like the concept units. Criteria A and B are next rehearsed in Units 02–05 and assessed cold-ish at Unit 06; D is not seen before Unit 08. **[[ALEX: decide — whether and where the A–D lite rehearsal is replaced — this plan deliberately does not solve it]]**

micro:bit inventory — fill in as boards go out

Quick tell: **V2** has a notch in the gold edge connector, a small microphone hole and LED by the logo, a touch-sensitive gold logo, and a speaker on the back. **V1** has a straight edge connector and none of those. V2 adds microphone, speaker and touch logo as inputs — this determines what later units can use. **[[ALEX: decide — Monday's 8A inventory never reached the vault — transfer both tables after this lesson]]**

No.	V1 / V2	Working?	Issued to	Notes
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				
12				
13				
14				
15				
16				
17				
18				
19				

After the lesson

- **Transfer the inventory** into the vault (kit note or Class Log) — the V1/V2 split and the working count shape every micro:bit lesson after this one.
- **The publish step is the one that must not get squeezed out.** A student who leaves without a published link has no submission route; the homework sheet tells them how to rebuild and publish from home, but check the stragglers.
- Flip the Class Log entry to **Taught**; log how far the class actually got (all to button A? all published?).
- **Check Google Classroom over the weekend** — seven days is short, so a nudge around Mon 7 Sep is worth more than a late one.
- If the download loop ate the hour, L2 opens there — the loop is the skill; everything after it can shift right.